Research Summary: Graphiti: Secure Graph Computation Made More Scalable

Paper by: Nishat Koti, Varsha Bhat Kukkala, Arpita Patra, Bhavish Raj Gopal (2024)

Hitarth Makawana
Roll No: 230479
hitarthkm23@iitk.ac.in

Ronav Puri
Roll No: 230815
ronavgp23@iitk.ac.in

April 9, 2026

Premise

The paper presents Graphiti, a highly scalable framework for secure graph computation that improves upon prior state-of-the-art systems (like GraphSC). It allows mutually distrusting parties to operate on graphs without leaking sensitive topological or node-level data.

- $\mathcal{O}(1)$ **Round Complexity**: Unlike previous frameworks whose round complexity scales linearly with the graph size $N = |V| + |E|$, Graphiti's message-passing round complexity is strictly constant and independent of N.
- **Decoupling Scatter**: The protocol fundamentally optimizes the message-passing phase by decoupling the traditional Scatter primitive into two distinct operations: Propagate (distributing data) and ApplyE (updating edges)
- **Linear Operations via Prefix Sums**: It completely removes the need for interactive oblivious selection (which requires heavy secure multiplications) by replacing them with non-interactive cumulative prefix sums over specifically ordered lists.
- **Applications**: Existing applications like *BFS, histograms and matrix factorisation, computing Pagerrank and its variants, graph convolutional network evaluation,* etc. can directly be improved by using Graphiti as the underlying secure graph computation framework.

Ideal Functionalities and Sub-Protocols Used

$\mathcal{F}_{Shuffle}$ - Secure Shuffling Functionality

- **Functionality:** Takes as input a secret-shared vector from the computing parties, applies a random secret permutation π, and returns the secret-shared shuffled vector.
- **Implementation**: Graphiti introduces a novel $(2+1)$-Shuffle protocol tailored for the 2-party setting with a trusted helper (preprocessing), executing in exactly 1 online round.

$\mathcal{F}_{insecSort}$ - Insecure Sorting Functionality

- **Functionality:** Generates a public mapping (permutation) that transitions a randomly shuffled DAG-list into a specific sorted order. Because the input array is already randomly shuffled by $\mathcal{F}_{Shuffle}$, revealing this mapping leaks no topological information.

Secure Shuffling Protocol: $(2+1)$-Shuffle (Online Phase)

To efficiently transition between the vertex, source and destination ordering, Graphiti requires a secure shuffle. The authors design a novel $\Pi_{(2+1)-Shuffle}$ protocol for two computing parties (P_0, P_1) and one trusted helper (P_2) that operates strictly in the offline preprocessing phase.

- **Input:** Additive shares $\langle T\rangle_0, \langle T\rangle_1$ of table T.
- **Preprocessing:** The helper party P_2 samples the composite permutations and generates random masking vectors (R_0, R_1, B_0, B_1). These are distributed to P_0 and P_1 offline.
- **Output:** Shares of the shuffled table $\langle T_O\rangle_0, \langle T_O\rangle_1$ where $T_O = \pi(T)$.
- **The Protocol:**
  1. P_1 computes and sends $A_0 = \pi_1(\langle T\rangle_1 + R_1)$ to P_0.
  2. P_0 computes and sends $A_1 = \pi'_0(\langle T\rangle_0 + R_0)$ to P_1.
  3. P_0 computes $\langle T_O\rangle'_0 = \pi_0(A_0)$ and sets $\langle T_O\rangle_0 = \langle T_O\rangle'_0 - B_0$.
  4. P_1 computes $\langle T_O\rangle'_1 = \pi'_1(A_1)$ and sets $\langle T_O\rangle_1 = \langle T_O\rangle'_1 - B_1$.
  5. P_0 and P_1 locally sample a shared random permutation π_2 and apply it to their respective shares to mask the final output from the helper.
- **Communication Complexity:** $2N\ell$ bits (exactly 1 message of size $N\ell$ per party in the online phase).

Graphiti Algorithm and Framework

DAG-List Representation

The graph $G(V, E, data)$ is represented as a list, called the DAG-list, which combines both vertices and edges into a single unified structure. This ensures the topology remains hidden during linear scans.

- Each entry is encoded as a tuple: **(src, dst, isV, data)**.
- A node u is encoded as *(u, u, 1, data)*.
- A directed edge $e(u, v)$ is encoded as *(u, v, 0, data)*.
- The data component is logically split during message passing into $data_s$ (data to send), $data_r$ (data received by an edge), and $data_g$ (data gathered by a node).

Core Message-Passing Primitives

- **Propagate:** A node propagates its state data to its outgoing edges. Through this operation, each directed edge $e(u, v)$ updates its data component as $e.data_r = e.data_r \parallel u.data_s$.
- **ApplyE (Apply-Edge):** All edges locally update their data by applying a user-defined function f_{AE}.
- **Gather:** A destination node aggregates the propagated data from all its incoming edges using a linear aggregation function $\oplus$. $v.data_g = v.data_g \parallel \bigoplus_{\forall e(u,v)} e.data_r$.
- **ApplyV (Apply-Vertex):** All nodes locally update their state data by applying a user-defined function f_{AV} based on the aggregated results.

Characteristics

- **Data-Oblivious Prefix Sums**: Edge propagation and node aggregation are achieved using a cumulative sum. Because addition is a linear operation in secret sharing, this requires zero online interaction.
- **Structure**: Each message-passing iteration consists of:

Algorithm 1 Propagate

Input: DAG-list G in *vertex order*
Output: Secret-shared DAG-list where node data is propagated to outgoing edges
Complexity Per Party: $O(1)$ rounds

for $i = |V|$ to 2 **do**
  $G[i].data_r \leftarrow G[i].data_s - G[i-1].data_s$
end for
Transition to the source order of the DAG-list via $\mathcal{F}_{Shuffle}$ and public mappings.
for $i = |N|$ to 1 **do**
  $G[i].data_r \leftarrow \sum_{j=1}^{i} G[j].data_r - G[i].data_s$
end for

Algorithm 2 Gather

Input: DAG-list G in *destination order*
Output: Secret-shared DAG-list where incoming edge data is aggregated into destination nodes

for $i = 1$ to N **do**
  $G[i].data_g \leftarrow \sum_{j=1}^{i} G[j].data_r$
end for
Transition to the vertex order of the DAG-list via $\mathcal{F}_{Shuffle}$ and public mappings.
for $i = |N|$ to 2 **do**
  $G[i].data_g \leftarrow G[i].data_g - G[i-1].data_g$
end for

Implementation & Results

- **Implementation Settings**: Developed in C++ leveraging the EMP-toolkit, executed over a simulated 1 Gbps LAN on AMD Ryzen Threadripper PRO servers.
- **Scheme**: 2-Party Computation (2PC) over rings with a Trusted Helper operating strictly in the offline preprocessing phase (secure against a semi-honest adversary).
- **Results**: The framework achieved massive scalability. Running a 10-hop Breadth-First Search (BFS) for contact tracing on a graph with 10^7 elements completed in under 2 minutes. This yielded up to a 1034x speedup in online runtime and a 106x reduction in communication costs compared to the GraphSC baseline.

Improving Concrete Efficiency

- **Optimized ApplyV**: Because Graphiti always returns the DAG-list to the vertex Order at the end of Gather, the non-linear ApplyV step (e.g., threshold comparisons) only needs to be executed on the first $|V|$ elements, rather than the entire list of size N.
- **Amortized Shuffling:** The mappings between vertex, source and destination orders remain static across iterations. These transitions are pre-computed during an initialization phase, meaning multiple message-passing rounds simply reuse the same permutations.
- **Minimal Data Reordering:** Instead of reshuffling the entire DAG-list tuples (which include static topological metadata), Graphiti only reorders the specific data payloads during transitions, massively reducing communication bandwidth.